Adaptive aggregation for subexponential SAT

A review of fifteen refinement packets, sharper time–space bounds,
and a measured finite-precision Cauchy-kernel improvement

Research prepared for Vladimir Reshetnikov

October 26, 2026

Abstract

We review the current ProveIt SAT research tree and all eight second-round packets received through `docs/incoming`, a total of fifteen refinement packets and two primary manuscripts. The strongest displayed general polynomial-space contraction coefficient is 76.90899, but combining already available ingredients gives 66.60514, and approaching their parameter boundary gives 66.19272. We use this stronger comparison baseline.

The principal new combinatorial step chooses a partition of nine groups into three triples separately for each exact mode-weight allocation. An averaging argument bounds the crossing-index sum, so direct enumeration improves the fixed-partition matrix-multiplication certificate. Relative to the reviewed contraction framework, the resulting normalized bounds are

$$T \le \mathrm{poly}(s+\mathcal{L})\, 2^{(36.75031252384+o(1))s/\sqrt{\log_2 s}}, \qquad S \le \mathrm{poly}(s+\mathcal{L})\, 2^{(34.02417800750+o(1))s/\sqrt{\log_2 s}}.$$

A separate recursive separator summation gives coefficient 55.70947831891 while retaining polynomial space. A low-degree specialization of the cached algorithm gives time coefficient 15.3919331988. These constants are per normalized Boolean wire, not per SAT variable. The logarithmic order of the general SAT bound is unchanged.

We also implement the existing joint Cauchy factorization directly modulo 2^p, using separate exact valuations and bounded unit residues. On six paired local-kernel fixtures it is 6.22–58.62 times faster than a strengthened rational-recurrence control with the same mode cutoff. Complete code, exact checks, raw timings, source hashes, and this LaTeX source accompany the report. No complete SAT solver or new proof-assistant verification is claimed.

Contents

1 Result, comparison standard, and proof boundary **3**
1.1 Normalized interface . . . 3
1.2 What has and has not been proved here . . . 4

2 Frozen source review **4**
2.1 Primary manuscripts and seven earlier packets . . . 4
2.2 All eight second-round packets . . . 5
2.3 The stronger combined baseline . . . 6

3 The shared arithmetic and recursion interfaces **6**
3.1 Asymmetric padding and side provenance . . . 7
3.2 Sharp precision with an additive buffer . . . 7
3.3 The full-orientation joint Cauchy basis . . . 8

4 Entropy of exact pair-weight lists **8**
4.1 Exact indexing and partition selection are computable . . . 9

5 The new adaptive aggregation step **10**
5.1 Evaluate children once, then build block tables . . . 10
5.2 Directly sum the crossing indices . . . 10

6 A uniform integer schedule and the global theorem **11**
6.1 Padding, degree, and depth without circular assumptions . . . 11
6.2 Solving the paid recursion . . . 12

7 Recursive separator summation in polynomial space **12**

8 A lower-degree cached specialization **14**

9 A finite-precision implementation of the known Cauchy basis **15**
9.1 Why no invalid division of a residue occurs . . . 15
9.2 Reusing inverses and resource cost . . . 16
9.3 Fair controls . . . 16

10 Measured results and exact verification **16**
10.1 Primary and independent exact checks . . . 17

11 Consequences for SAT and remaining limits **18**

12 Reproducible deliverable and integration **18**

1 Result, comparison standard, and proof boundary

The task is to improve on the combined frontier, rather than repeat an idea already in another packet. Three contributions survive that comparison:

1. **Adaptive aggregation.** For every exact mode-weight allocation, choose the least expensive crossing partition among the 280 partitions of nine groups into triples. The new bound follows from a finite averaging identity, not a hypothetical compact tensor compiler.
2. **Polynomial-space separator summation.** Recursively split the mode graph and, after fixing the crossing indices, sum its two sides independently. Exact distributivity avoids jointly enumerating the internal indices of separate branches and reduces the paid recursive work.
3. **A practical arithmetic implementation.** Evaluate the known Cauchy features with bounded modular arithmetic, exact valuation bookkeeping, reused denominators, and streaming. Compare against both the unchanged archive and a stronger matched recurrence control.

1.1 Normalized interface

The common source interface is a bipartite Boolean tensor network. Its contraction is

$$Z = \sum_{x\in\{0,1\}^E} \prod_v T_v(x|_{\delta(v)}).$$

The root has at most s wires; the initial precision and truncated coefficient axis have size $O(s)$ under the source normalization. The root maximum degree is at most $\lceil s^{9/10} \rceil$, the tensor descriptions have total length $\mathcal{L}$, and original tensor entries obey the source's permitted $\exp(O(s^{9/10}))\,\mathrm{poly}(s+\mathcal{L})$ evaluation cost and polynomial-space specification. Arithmetic takes place in an unramified extension of the two-adic integers, optionally with a truncated nilpotent coefficient axis, modulo a prescribed power of two. All scalar and coefficient dimensions remain polynomial in s.

These are the same conditions under which the compared normalized constants are derived. An arbitrary dense tensor supplied by an arbitrary costly oracle does not automatically satisfy them. Tensor creation precision is part of its immutable description: later evaluation reuses the specified canonical representatives. Canonical lifting is not assumed to be a ring homomorphism.

Theorem 1.1 (Adaptive time–space certificate)**.** *Under the normalized interface and the inherited exact identities specified in Section 1.2, there is an exact contraction algorithm with*

$$T(s) \le \mathrm{poly}(s+\mathcal{L})2^{(C_{\mathrm{ad}}+o(1))s/\sqrt{\log_2 s}}, \tag{1}$$

$$S(s) \le \mathrm{poly}(s+\mathcal{L})2^{(C_{\mathrm{mem}}+o(1))s/\sqrt{\log_2 s}}, \tag{2}$$

$$C_{\mathrm{ad}} = \frac{\pi\sqrt{40\log_2 9}}{\ln 2}\frac{\sqrt{42}}{9} = 36.75031252383623\ldots,$$

$$C_{\mathrm{mem}} = \frac{\pi\sqrt{40\log_2 9}}{\ln 2}\frac{2}{3} = 34.02417800749051\ldots.$$

There is also a polynomial-space implementation with coefficient $C_{\mathrm{ps}} = 55.70947831890997\ldots$, given explicitly in Section 7.

The two algorithms are different resource choices. The faster certificate stores subexponential tables and gives up polynomial space. It does not dominate the polynomial-space branch under a fixed memory budget. The $o(1)$ terms and eventual parameter thresholds matter: a coefficient such as 36.7503 is not an observed running-time constant or a finite-size guarantee without its correction term.

1.2 What has and has not been proved here

The adaptive partition identity, the streamed separator algorithm, their resource accounting, and the new schedule analysis are proved below. The new arithmetic implementation has an explicit algebraic correctness argument and independent exact finite checks.

The following are imported from the reviewed sources, with their hypotheses preserved: the deformation and exact recovery identities; construction of the child tensor networks; the two-stage asymmetric packing lemma; the sharp Hermite precision loss; the all-level, group-preserving joint Cauchy basis; the original entry-oracle and canonical-representative conventions; and the reductions from SAT. This report does not independently reconstruct all of those arguments or rebuild the upstream Lean development. Consequently the global SAT consequences are *relative refinements of that framework*, not a newly certified independent proof of the ETH claim.

Finite checks do not prove the asymptotic theorem. Conversely, a smaller proved upper-bound coefficient does not establish a speedup over the actual runtime of an unimplemented algorithm. The measured claim concerns the identified feature-table kernel only.

2 Frozen source review

The closing repository snapshot is `474ddba696a5d385c091c23fb2995f69edd13e00`. The initial review used `ab2fbc9e03d5c3d0ad6dec8686960bfe498b0b06`, where seven new SAT archives had just arrived in `3e57705cf14ee`. A closing check found one additional matching-potential packet in `d098ba436b33` and the filing commit `6d3adc450615`, committed October 26, 2026 at 19:03:44 Pacific time. All eight second-round packets are now under `Computability/SubexponentialSAT/Research`; their historical arrival archives remain identified in the ledger.

The closing review includes the additional matching report and verifies the filed versions of the other seven against the reviewed archives. The final corpus contains fifteen refinement packets and two primary manuscripts. The initial seven-packet summary was therefore insufficient, and the report uses the expanded closing corpus.

The first relocation commit is `2b925d8800cd`; the original preprint was vendored by `3e096089911f`. Its upstream pin is `7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`. The accompanying `provenance/reviewed_sources.json` records the closing file paths, Git blob hashes, SHA-256 hashes, and immutable URLs. The initial source and archive inventory is retained separately. No repository branch or file was submitted or changed.

2.1 Primary manuscripts and seven earlier packets

The OpenAI manuscript's Appendix D gives the general order

$$\mathrm{poly}_k(n,m)2^{O_k(n\log\log n/\sqrt{\log n})}, \tag{3}$$

and the fixed-density order $2^{O_{k,D}(n/\sqrt{\log n})}$, in polynomial space [1]. The separate Claude manuscript states an order based on $\min\{km/\sqrt{\log m}, k^2 n\log\log n/\sqrt{\log n}\}$ [9]. Its implementation and matched explicit network constant are not provided. Numerical domination over that different architecture cannot be inferred from a smaller normalized contraction coefficient.

Earlier packet	Established ingredient and remaining limitation
`cauchy_mode_improvement`	Integral factorial Cauchy features with weights $1 + b + 2\nu_2(b!)$; slope three improves the mode-entropy coefficient. No change in logarithmic order.
`eth_contraction_improvements`	Sharp taper/precision, leading precision $6MD/u$ in place of $24MD/u$; explicit coefficient 648.44107; streamed finite-difference extraction.
`3sat_precision_refinement`	Exact recovery loss and a solved real taper minimax problem. The $n \log\log n/\log n$ target requires a missing structured summation algorithm.
`sat_density_improvement`	Uniform density-sensitive exponent $O_k(n \min\{\log(2 + m/n), \log\log n\}/\sqrt{\log n})$. It improves some intermediate-density regimes.
`eth_density_refinement`	Overlapping density-sensitive bound; precision refinements and a local Smith-spectrum analysis. Neither changes the unrestricted denominator $\sqrt{\log n}$.
`3sat_refinement`	Scalar parity lift, reduced incidence-degree cap, and removal of an auxiliary coefficient axis. Its second-contraction target needs an unsupplied compact factorization.
`3sat_asymptotic_analysis`	Restricted barriers for taper and mode retuning. Stronger targets are conditional on missing recursive or linear-wire interfaces.

These reports are filed under `Computability/SubexponentialSAT/Research`. Their exact file identities are in the provenance ledger; the independent review is included as `audit/frontier_audit.md`.

2.2 All eight second-round packets

The following numeric coefficients use the common scale $s/\sqrt{\log_2 s}$ in a base-two exponent, under the general degree premise. They must not be mixed with lower-degree specializations.

Second-round packet	Coefficient	Main ingredient
`eth_balanced_packing`	94.74820	Nearly lossless balanced padding and an additive precision guard.
`eth_high_moment`	94.74820	Higher-moment labeling and an alternative orbit-padding construction.
`eth_packet_refinement`	95.66394	Improved pair loads, crossing fraction, and a thin precision gap.
`eth_recursive_refinement`	133.20737	Sharper packing and an actual-domain Cauchy refinement.
`3sat_round2_refinement`	84.06627	Matching-potential packing and a combined two-stage schedule.
`eth_asymmetric_packet_balancing`	76.90899	Different bounds for the retained and fresh tensor sides.
`eth_matching_refinement`	94.74820	Exact matching-potential labeling and thin additive precision gaps; no new summation method.

Second-round packet	Coefficient	Main ingredient
`eth_joint_cauchy`	319.68576	Full-orientation slope-four modes at all retained levels; unchanged family multiplicity.

All are polynomial-space certificates as displayed in the table. The last packet also gives a cached, fixed-partition Strassen branch with time coefficient 225.59706 and space coefficient 196.19945. The asymmetric packet's 31.06413 specialization assumes root degree $s^{1/12}$, rather than $s^{9/10}$. The high-moment and recursive reports also give lower-degree variants. These are retained as separate branches in the comparison, not treated as weaker general algorithms.

2.3 The stronger combined baseline

The eight second-round packets were each written against the older seven-packet frontier. Combining the new archives with one another is therefore necessary. Asymmetric packing preserves the old child graphs and gives

$$M_{\mathrm{child}} \le (2/r + o(1))M, \qquad c_i \le (2(1 - 1/r) + o(1))M/D_i.$$

The joint Cauchy basis has exactly the same mode-family multiplicity and source/target group locality. Its weights have slope four instead of three. It can therefore be composed with the asymmetric schedule.

For the existing $r = 16$ schedule, that composition gives

$$C_{\mathrm{comp}} = \frac{\pi\sqrt{15/8}}{\ln 2}(9 + \sqrt{3}) = 66.60514340761482\ldots. \tag{4}$$

Letting the parameters approach the admissible boundary slowly enough to absorb their logarithmic buffers gives

$$C_{\mathrm{lim}} = \frac{\pi\sqrt{150}}{\ln 2}\left(1 + \frac{1}{3\sqrt{3}}\right) = 66.19272369357722\ldots. \tag{5}$$

The complete integer boundary schedule is reproduced in the audit. The new polynomial-space coefficient is compared against (5), not just against 76.90899. A preliminary two-alphabet retuning also gives 66.12053; its complete schedule and independent review appear in `audit/two_alphabet_review.md`. The separator summation proved below improves substantially on both.

Similarly, the cached fixed-partition evaluator can be combined with the better packing and joint modes. For instance $r = 8$, block sizes $(2, 3, 3)$, and geometric taper ratio $1/2$ give the limiting coefficient

$$C_{\mathrm{fixed},8} = \frac{\pi\sqrt{120}}{\ln 2}\sqrt{\frac{30 + 12(\log_2 7 - 2)^2}{64}} = 38.16774569701537\ldots. \tag{6}$$

Thus a comparison only with the packet's displayed 225.59706 would greatly overstate the contribution. The adaptive evaluator improves this stronger composed certificate as well. We do not claim to have optimized every possible algorithm or every unreported constant in the corpus.

3 The shared arithmetic and recursion interfaces

At an active call with M wires, let $L = 2^a$, let r be the group alphabet, and write $Q = r^a$. Label each side of the graph by $[r]^a$ and pad every ordered label pair to a common multiple H_{pad} of L. The total padded mass is $P = H_{\mathrm{pad}}Q^2$.

3.1 Asymmetric padding and side provenance

The reviewed asymmetric packing lemma gives a deterministic polynomial-time labeling with pair load bounded by

$$\left(1+\frac{2}{h-1}\right)\left(\frac{M}{Q^2}+\frac{R}{Q}\right)+2(hC+1)\ln Q, \tag{7}$$

where R, C bound row and column degrees, respectively, and $h \geq 2$. It first balances whole row vertices greedily, then derandomizes column labels by an exact rational conditional expectation. Parallel edges are distinct ports and are covered by the lemma.

Taking $h \asymp \log s$ and rounding upward to a packet multiple yields $P = (1+o(1))M$ whenever, uniformly over active calls,

$$\frac{QR}{M} = o(1), \qquad \frac{Q^2C(\log s)^2}{M} = o(1), \qquad \frac{LQ^2}{M} = o(1). \tag{8}$$

The source orientation construction is unchanged. A child has at most $2P/r$ wires. All retained parent tensors lie on one side of a child; fresh packet tensors lie on the other and have degree at most $2L$. Hence below the root we may keep

$$R_d \leq \Delta, \qquad C_d \leq 2L_{d-1}, \qquad \Delta = \left\lceil s^{9/10} \right\rceil, \tag{9}$$

provided newly created degrees stay below Δ.

3.2 Sharp precision with an additive buffer

For $i = \nu_2(t) < u \leq a$, put $D_i = 2^{a-i}$ and use multiplicities $q_i = \rho(u-i)$, where

$$\rho = \left\lceil \frac{M(2L-1)+1}{L(2u-2+2^{1-u})} \right\rceil.$$

The denominator is the exact number of Hermite conditions per unit taper. The sharp recovery formula permits the sufficient precision

$$p_t = p + 6\rho D_i + \rho u(a+1). \tag{10}$$

This is an imported result; its leading constant six is already optimized within the earlier full-loss taper model.

Let $\lambda = \lceil \log_2 s \rceil$, stop at $M \leq H = \lceil s/\lambda \rceil$, and set

$$b = \lceil 6 \log_2 \lambda \rceil, \qquad a_d = \sum_{j=0}^{d} u_j + (d+1)b. \tag{11}$$

For active depths $O(\log \lambda)$, ancestral precision obeys $p = O(s\lambda^2 2^{a_{d-1}})$. Meanwhile $D_i \geq 2^{a_{d-1}+b+1}$. Since $s/M \leq \lambda$ and $u_d \leq \lambda$,

$$\frac{p}{MD_i/u_d} = O(\lambda^4 2^{-b}) = o(1).$$

The final summand of (10) is also lower order. When the least active u_d tends to infinity, uniformly,

$$p_t \leq (6+o(1))MD_i/u_d. \tag{12}$$

3.3 The full-orientation joint Cauchy basis

For $R = 2^i$, $N = L/2$, and

$$s_j = (j \bmod R) + 2R\lfloor j/R \rfloor, \qquad 0 \le j < N,$$

one crossing orientation has the scalar difference kernel

$$W_{xy}(T) = \frac{\Gamma(T)}{T + x - y}, \quad \Gamma(T) = \frac{\prod_{v=-L}^{L-1}(T - v)}{\mathcal{C}'(0)}, \quad T = t + 2^{a+1}\zeta.$$

Here $\mathcal{C}(T) = \prod_{v=-L}^{L-1}(T - v)$, $\nu_2(T) = i$, and $\Gamma(T) \in 2^{a+1}\mathcal{O}$. The displayed quotient is interpreted after cancelling its linear polynomial factor.

Define

$$P_b(x) = \prod_{h<b} \frac{x - s_h}{T + x - s_h}, \qquad Q_b(y) = \prod_{h<b} \frac{s_h - y}{T + s_h - y},$$
$$f_b(x) = \frac{TP_b(x)}{T + x - s_b}, \qquad g_b(y) = \frac{TQ_b(y)}{T + s_b - y}.$$

The exact finite telescoping identity is

$$\frac{1}{T + x - y} = \frac{1}{T} \sum_{b=0}^{N-1} f_b(x) g_b(y). \tag{13}$$

For $b = qR + r'$, $0 \le r' < R$, put $E_b = q + \nu_2(q!) = 2q - \mathrm{popcount}(q)$. The prior integrality lemma shows that $F_b = 2^{-E_b} f_b$ and $G_b = 2^{-E_b} g_b$ are integral on the slot domain. Thus

$$W_{xy}(T) = \sum_b A_b(x) G_b(y), \qquad A_b(x) = \frac{\Gamma(T)}{T} 2^{2E_b} F_b(x), \tag{14}$$

with certified weight

$$w_q = a + 1 - i + 4q - 2\,\mathrm{popcount}(q). \tag{15}$$

There are R modes of each q per orientation. Each orientation's source form reads a whole half-packet, but all those ports still belong to a single source group; its targets belong to one other group. This is exactly the locality needed for the unchanged child factorization.

For any ordered pair of distinct groups g, h, uniform label-pair padding therefore gives at most

$$c_{gh} = \frac{2P}{r^2 D_i} = \left(\frac{2}{r^2} + o(1)\right) \frac{M}{D_i} \tag{16}$$

modes at each index q. Summing over $r(r-1)$ pairs recovers the asymmetric packet's total family envelope. No extra valuation-class factor is introduced.

4 Entropy of exact pair-weight lists

The recursion sums only mode tuples whose total weighted degree is below p_t. Split a tuple by the exact weights b_{gh} allocated to its directed group pairs, with $\sum_{g \ne h} b_{gh} < p_t$. For fixed r, there are only polynomially many such arrays, although the polynomial exponent is large.

For a particular array, let $\mathcal{I}_{gh}$ be the list of mode tuples on that pair with exact weight b_{gh}, and put $h_{gh} = \log_2 |\mathcal{I}_{gh}|$. Arrays with an empty list contribute zero and are skipped. Each child factor Z_g depends only on indices incident with g. Define the convenient scale

$$\mathcal{K}(M, u) = \frac{\pi\sqrt{2}}{\ln 2} \frac{M}{\sqrt{u}}. \tag{17}$$

Lemma 4.1 (Entropy on selected directed pairs)**.** *For every fixed set I of directed pairs,*

$$\sum_{e \in I} h_e \le \left(\frac{\sqrt{|I|}}{r} + o(1) \right) \mathcal{K}(M, u), \tag{18}$$

uniformly over exact weight arrays and active calls in the schedules below. More generally, fixed nonnegative weights v_e give the factor $\sqrt{\sum_e v_e^2}/r$ on the right.

Proof. Lower the actual weights to $\widehat{w}_q = 1 + 4q - 2\,\mathrm{popcount}(q)$ and extend the finite index range to all $q \ge 0$, both valid upper bounds. The counting function of these weights is $x/4 + O(\log(x+2))$, giving

$$\sum_{q \ge 0} -\ln(1 - e^{-\theta \widehat{w}_q}) = \frac{\pi^2}{24\theta} + O(\log^2(2/\theta)).$$

This follows by integrating against the counting function and using $\int_0^\infty -\ln(1 - e^{-x})\,dx = \pi^2/6$.

Use the generating-product bound for pair e with parameter θ/v_e when $v_e > 0$, multiply its logarithm by v_e, and sum. With $c = (2/r^2 + o(1))M/D_i$, this yields

$$\ln 2 \sum_e v_e h_e \le \theta p_t + \frac{c\pi^2}{24\theta} \sum_e v_e^2 + O(c \log^2(2/\theta)).$$

Optimizing the leading two terms and applying (12) proves the claim. The saddle parameter is of order $\sqrt{u}/D_i$. The additive buffer ensures $D_i \ge \lambda^6$ up to a fixed factor, so the error divided by $M/\sqrt{u}$ tends to zero uniformly. This joint argument also covers very small or zero individual allocated budgets; it does not assume a separate asymptotic saddle for every pair. □

4.1 Exact indexing and partition selection are computable

For mode weights $w_1, \ldots, w_v$, the finite dynamic program

$$D(j, z) = D(j-1, z) + D(j, z - w_j), \quad D(0, 0) = 1,$$

with other zero/negative boundary values counts exact-weight tuples. The number of modes and p_t are polynomial in s, and the counts have polynomial bit length, since they are at most $(p_t + 1)^v$. The table provides exact list sizes, rank/unrank operations, and enumeration with polynomial delay. No counting oracle is assumed.

The partition algorithm below minimizes a product of these integer list sizes. It can compare the products exactly, without calculating floating-point logarithms. If binary ranks are used internally, rounding each logarithm adds at most a constant per pair and does not change the leading coefficient.

5 The new adaptive aggregation step

We now fix $r = 9$. For each exact weight array, consider all 280 unlabelled partitions of the nine groups into three blocks of size three. Choose one minimizing

$$H_{\text{cross}} = \sum_{g,h \text{ in different blocks}} h_{gh}. \tag{19}$$

The choice is allowed to depend on the weight array. Once chosen, it is fixed while evaluating that array.

Lemma 5.1 (Adaptive crossing bound)**.** *For every nonnegative directed edge weighting h_{gh}, there is a partition into triples for which*

$$H_{\text{cross}} \le \frac{3}{4} \sum_{g \ne h} h_{gh}. \tag{20}$$

For the exact mode lists, this implies $H_{\text{cross}} \le (1/\sqrt{2} + o(1))\mathcal{K}(M, u)$.

Proof. In a uniformly random partition into triples, a fixed group has two partners among the other eight groups. Thus a fixed directed pair is separated with probability $6/8 = 3/4$. By linearity of expectation, the mean crossing weight is $3/4$ of the total; its minimum is no larger. The total has 72 directed pairs, so Lemma 4.1 bounds it by $(\sqrt{72}/9 + o(1))\mathcal{K}$. Multiplication by $3/4$ gives $1/\sqrt{2}$. □

5.1 Evaluate children once, then build block tables

For each group g, evaluate and store its factor Z_g on all its incident pair indices. Each evaluation is a real recursive contraction of its child; this is not a cheap black-box oracle. A group has 16 incident directed pairs, so the number of recursive calls per group is at most

$$2^{(\alpha + o(1))\mathcal{K}(M,u)}, \qquad \alpha = 4/9. \tag{21}$$

For a triple B, build a table indexed by the directed pairs joining B to its complement. Sum over its internal pair indices the product of the three cached group factors. There are

$$\underbrace{2 \cdot 9 \cdot 3 - 3^2 - 3}_{42} \text{ incident directed pairs}, \qquad \underbrace{2 \cdot 3 \cdot 6}_{36} \text{ boundary directed pairs}.$$

For every choice of the partition, Lemma 4.1 gives the bounds

$$\text{block construction work} \le 2^{(\beta + o(1))\mathcal{K}(M,u)}, \qquad \beta = \sqrt{42}/9, \tag{22}$$

$$\text{block-table storage} \le 2^{(j + o(1))\mathcal{K}(M,u)}, \qquad j = 2/3. \tag{23}$$

There are only three tables and nine caches, absorbed into polynomial factors.

5.2 Directly sum the crossing indices

After the three block tables have been built, summing their product over all crossing indices costs $2^{H_{\text{cross}}}$ table operations. By Lemma 5.1, its coefficient is at most $1/\sqrt{2}$, strictly less than β, since $1/2 < 42/81$. Thus the block construction is the dominant local nonrecursive work. No matrix multiplication algorithm, division by a nonunit, or unconstructed tensor compression is required.

Theorem 5.2 (Local adaptive recurrence)**.** *Up to polynomial factors per call, the adaptive evaluator satisfies*

$$T_d \le 9\,2^{(\alpha+o(1))\mathcal{K}_d} T_{d+1} + 2^{(\beta+o(1))\mathcal{K}_d}, \tag{24}$$

with $\alpha = 4/9$, $\beta = \sqrt{42}/9$. Polynomial factors in the number of recursive calls depend on s, independently of $\mathcal{L}$; a single fixed polynomial in $s + \mathcal{L}$ pays for entry reading and arithmetic. Its largest local table has coefficient $j = 2/3$ on scale $\mathcal{K}_d$.

Proof. The first term pays for every cached child contraction, and the second pays for block-table construction and the final crossing summation. Each original mode tuple belongs to one exact weight array. Group factors and block tables only regroup finite sums and products in a commutative ring, so their contraction equals the original recursion identity. Exact weight arrays, interpolation nodes, and auxiliary evaluation choices contribute polynomial factors, because $r = 9$ is fixed and all displayed budgets are polynomial. The preceding counts prove the bounds. □

Access cost. Choose the partition before constructing its group caches. Exponential tables are not assumed to offer free random access. Store records lexicographically in their pair indices. Cache layouts can be sorted into the order needed by the chosen partition at polynomial overhead per record. In a lexicographic enumeration, a projection onto a factor's axes advances, repeats, or rewinds a suffix. Charge each rewind to the complete suffix assignments just traversed; total movement is bounded by a polynomial factor times the full enumeration. A global lexicographic enumeration of crossing indices can use each block table in its induced order. The record lengths, number of axes, and logarithms of record counts are polynomial in s. Therefore the stated exponents apply to bit work, not only to an idealized unit-cost table-lookup model.

6 A uniform integer schedule and the global theorem

Let $\lambda = \lceil \log_2 s \rceil$ and define, using integer comparisons,

$$B_\lambda = \max\{j \ge 0 : 9^{20j} \le 2^\lambda\}, \quad U = B_\lambda - \left\lceil \sqrt{\lambda} \right\rceil, \quad u_d = \left\lfloor U/2^d \right\rfloor. \tag{25}$$

Use $r = 9$, the buffer (11), and $H = \lceil s/\lambda \rceil$. Handle bounded sizes or failed finite admissibility checks by direct contraction. The proof below shows that all those checks eventually succeed.

6.1 Padding, degree, and depth without circular assumptions

First consider every hypothetical active depth $d \le d_* := \lceil \log_4 \lambda \rceil + 2$ and every $M > H$. The geometric sum gives

$$a_d \le 2U + O((\log \lambda)^2), \qquad u_d \ge \lambda^{1/2+o(1)}.$$

Thus the floor errors are uniformly relative $o(1)$, and $U/\lambda \to 1/(20 \log_2 9)$.

At the root, $R = C = \Delta$. By definition of U,

$$2a_0 \log_2 9 + \tfrac{9}{10} \log_2 s - \log_2 s \le -2 \log_2 9 \sqrt{\lambda} + O(\log \lambda).$$

This tends to $-\infty$ fast enough to absorb the precision and padding logarithms. Below the root, (9) gives the same leading negative gap for $Q_d R_d / M$, since $a_d \le 2U + O((\log \lambda)^2)$. The other two padding gates have leading exponent at most

$$\frac{1 + 2 \log_2 9}{10 \log_2 9} - 1 < 0.$$

All newly created degrees are at most $s^{1/(10\log_2 9)+o(1)} < s^{9/10}$. This proves (8) uniformly over the hypothetical range.

Consequently the padded mass is $(1+o(1))M$, and eventually each child has at most $M/4$ wires, since $2/9 < 1/4$. An active path has at most $\log_4 \lambda + O(1)$ steps, validating the hypothetical range and closing the induction. Sharply,

$$M_d \le s\left((2/9)(1+\delta_s)\right)^d, \qquad \delta_s \longrightarrow 0. \tag{26}$$

The negative $\Theta(\sqrt{\lambda})$ slack is essential. Setting the parameters exactly on the boundary and then adding the logarithmic buffers would not justify nearly lossless padding. The inward retreat in (25) resolves this while preserving the limit coefficient.

6.2 Solving the paid recursion

Equations (17), (25), and (26) imply, uniformly,

$$\mathcal{K}_d \le (\kappa + o(1))\gamma^d \frac{s}{\sqrt{\lambda}}, \quad \kappa = \frac{\pi\sqrt{40\log_2 9}}{\ln 2}, \quad \gamma = \frac{2\sqrt{2}}{9}. \tag{27}$$

In fact, the packing relative error is $O(1/\lambda)$ plus terms that are exponentially small in $\sqrt{\lambda}$ or a fixed power of s. Over $O(\log\lambda)$ levels its product is $1+o(1)$. Equivalently one may keep $\gamma_s = \gamma + o(1)$ until the uniformly convergent geometric series is summed. A path that pays for child caches through depth $d-1$ and performs block work at depth d has coefficient at most

$$\kappa\left[\alpha\frac{1-\gamma^d}{1-\gamma} + \beta\gamma^d\right].$$

The bracket is a convex combination of $\alpha/(1-\gamma)$ and β. The exact inequalities

$$\frac{\alpha}{1-\gamma} = \frac{4}{9-2\sqrt{2}} < \frac{2}{3} < \frac{\sqrt{42}}{9} = \beta \tag{28}$$

therefore show that the root block term controls the entire recurrence. This proves the time coefficient $\kappa\beta$ in Theorem 1.1.

At any time, there are polynomially many recursion levels of resident caches and tables. Their sizes add rather than multiply; deeper coefficient bounds decrease geometrically. The largest local table is bounded by $2^{(j+o(1))\mathcal{K}_0}$, so the global space coefficient is κj.

The leaf cost is $2^{O(s/\lambda)}$. Original entry costs are $\exp(O(s^{9/10}))$. Fresh packet entry cost remains $\exp(O(L\log s))$, where $L = s^{1/(10\log_2 9)+o(1)}$ at most. All are $2^{o(s/\sqrt{\lambda})}$. Polynomial work at $O(\log\lambda)$ levels contributes only $O(\lambda\log\lambda)$ to the logarithm. This includes preprocessing, exact indexing, and sorting. The precision and ring dimensions remain polynomial by the buffer argument. This completes the theorem, including its bit-operation and memory accounting.

7 Recursive separator summation in polynomial space

The exact pair-weight allocation does more than enable caches. It turns the retained mode sum into a Cartesian product of pair lists, allowing independent internal sums to be computed without storing them. This uses ordinary separator conditioning and distributivity, in the tradition of recursive conditioning [10]; the new contribution is its explicit paid application and coefficient bound in this contraction framework.

Fix an exact weight array and a binary tree of the r group vertices. At a node containing a group set B, split B into two nearly equal parts. For each assignment to the directed pairs crossing that split, recursively sum each side over its own internal pairs and multiply the two answers. Assignments to pairs already fixed by ancestor splits remain parameters. At a leaf $\{g\}$ evaluate the original factor Z_g by a real recursive child contraction. Accumulate one sum and product at a time.

Lemma 7.1 (Exact streamed separator identity)**.** *The algorithm evaluates the original finite mode sum using polynomial space. If E_ℓ is the union of cut-pair sets on a root-to-leaf path, then the number of expensive group-factor evaluations is at most*

$$\sum_{\ell=1}^{r} \prod_{e\in E_\ell} |\mathcal{I}_e|. \tag{29}$$

Proof. For a split $B = B_1 \sqcup B_2$, fixing its crossing indices separates all remaining internal variables into two disjoint sets. No factor in B_1 depends on an internal index of B_2, or conversely. Finite distributivity therefore gives a product of the two conditional sums, followed by summation over the crossing indices. Induction on the tree proves exactness.

A leaf is visited once for each assignment to the successive cut sets on its path. Those cut sets are disjoint, and their Cartesian product has the size in (29). Internal indices of sibling branches are summed independently; their domain sizes add to the work rather than multiply through the leaf. Each leaf evaluation pays for Z_g explicitly. No free group oracle is assumed. All internal-node overhead is bounded by a fixed polynomial times this call count. The depth is $O(\log r)$, and the assignment ranks and ring accumulators have polynomial bit size, proving the space bound. □

Lemma 7.2 (Balanced-tree cut profile)**.** *For a balanced binary split on r groups, the largest number of directed pairs on a leaf path is $v(r)$, where*

$$v(1) = 0, \qquad v(r) = 2\,\lfloor r/2\rfloor\,\lceil r/2\rceil + \max\{v(\lfloor r/2\rfloor), v(\lceil r/2\rceil)\}. \tag{30}$$

For $r = 2^j$, $v(r) = 2(r^2-1)/3$. In particular $v(16) = 170$, whereas the full directed group graph has 240 pairs.

Proof. The root split contributes $2\,\lfloor r/2\rfloor\,\lceil r/2\rceil$ directed pairs to every leaf path. A path then enters one child, proving the recurrence. For powers of two it becomes $v(r) = r^2/2 + v(r/2)$; summing the geometric series gives $2(r^2-1)/3$. □

Applying Lemma 4.1 separately to each of the fixed r path sets in (29) gives the local recurrence

$$T_d \le \mathrm{poly}(s)\, 2^{(\beta_{\mathrm{str}}(r)+o(1))\mathcal{K}_d} T_{d+1}, \qquad \beta_{\mathrm{str}}(r) = \frac{\sqrt{v(r)}}{r}. \tag{31}$$

A single common fixed polynomial in $s + \mathcal{L}$ pays for entry reading and arithmetic. This is a reduction in the number of paid child evaluations, not an assumption that the mode graph can be solved in subexponential time in its index length. It still has exponential work on that length.

Theorem 7.3 (Polynomial-space coefficient below 56)**.** *Under the general normalized interface, streamed balanced binary summation with $r = 16$ gives*

$$T \le \mathrm{poly}(s+\mathcal{L}) 2^{(C_{\mathrm{ps}}+o(1))s/\sqrt{\log_2 s}}, \qquad C_{\mathrm{ps}} = \frac{5\pi\sqrt{17}}{2\ln 2}\left(1+\frac{1}{3\sqrt{3}}\right) = 55.70947831890997\ldots, \tag{32}$$

using polynomial space.

Proof. Set $g = \lceil \sqrt{\lambda} \rceil$,

$$B = \left\lfloor \frac{\lambda - g}{80} \right\rfloor, \qquad u_0 = B, \qquad u_d = \lfloor 3B/4^d \rfloor \quad (d \geq 1),$$

and keep the same buffer $a_d = \sum_{j \leq d} u_j + (d+1)b$ and leaf threshold. The total retained range is at most $2B$.

To verify applicability, first consider hypothetical active depths $d \leq \lceil \log_6 \lambda \rceil + 2$. Uniformly there, $u_d \geq \lambda^{1-\log_6 4+o(1)} \to \infty$. At the root the $Q^2\Delta$ exponent is at most $\lambda - g/10 + O(\log \lambda)$. Below it the retained-row $Q\Delta$ exponent is at most $\lambda - g/10 + O((\log \lambda)^2)$. Fresh-side and rounding exponents are at most $9(\lambda - g)/40 + o(\lambda) < \lambda$. All padding gates hold; fresh packet degrees remain below Δ. Nearly lossless padding gives eventual child ratio at most $1/6$, closing the depth bootstrap, and sharp ratio $(1 + o(1))/8$.

The sum of $M_d/\sqrt{u_d}$ along a path is bounded by

$$\left[\sqrt{80} \left(1 + \frac{1}{3\sqrt{3}} \right) + o(1) \right] \frac{s}{\sqrt{\lambda}}.$$

Multiply by $(\pi\sqrt{2}/\ln 2)\beta_{\mathrm{str}}(16)$ with $\beta_{\mathrm{str}}(16) = \sqrt{170}/16$. This gives (32). Entry, precision, interpolation, and leaf costs are lower order as in the preceding theorem. The separator recursion, exact pair indexing, and network recursion are all streamed depth first, so only polynomial space is retained. □

Relative to the combined $r = 16$ polynomial-space boundary certificate (5), the coefficient is multiplied by

$$\sqrt{\frac{170}{240}} = \sqrt{\frac{17}{24}} = 0.8416254115\ldots.$$

This is a 15.84% reduction in that leading upper-bound coefficient. It is an exponential separation between the displayed certificate functions, not a measured SAT speedup. No claim of global optimality of the separator shape or alphabet is needed. The earlier two-alphabet retuning is also exceeded by this simpler $r = 16$ construction.

8 A lower-degree cached specialization

For bounded-degree clause networks it is useful to keep the root's stronger structural premise instead of replacing it by $s^{9/10}$.

Corollary 8.1 (Lower-degree time–space bound)**.** *If the root maximum degree is at most* $\lceil s^{1/12} \rceil$*, the adaptive cached algorithm has time and space coefficients*

$$C_{\mathrm{low}} = \frac{\pi}{\ln 2} \sqrt{\frac{100 \log_2(162)}{33}} \frac{\sqrt{42}}{9} = 15.3919331988\ldots, \tag{33}$$

$$C_{\mathrm{low,mem}} = \frac{\pi}{\ln 2} \sqrt{\frac{100 \log_2(162)}{33}} \frac{2}{3} = 14.2501611298\ldots. \tag{34}$$

The coefficients remain per normalized wire. The original permitted tensor entry-cost premise is unchanged.

Proof. Let $q = 17/50$ and define

$$B_\lambda = \max\{j \ge 0 : 162^j \le 2^\lambda\}, \quad W = B_\lambda - \left\lceil \sqrt{\lambda} \right\rceil, \quad u_d = \left\lfloor \frac{33}{50} W \left(\frac{17}{50}\right)^d \right\rfloor.$$

The total leading retained budget is at most W, with limiting fraction $A = 1/\log_2(162)$. The maximum old degree at depth d is bounded by $\max\{\left\lceil s^{1/12} \right\rceil, 2L_{d-1}\}$: packet lengths increase with depth, and retained tensors do not gain ports.

If the original-degree term dominates, the stronger symmetric packing constraint has exponent at most $2\log_2(9)A + 1/12 < 1$, because $1/12 < A$. If the created-degree term dominates, its exponent is at most $(1 + 2\log_2 9)A = 1$, with the required negative $\Theta(\sqrt{\lambda})$ slack. The rounding gate has the same bound since $1 + 2\log_2 9 = \log_2(162)$. All additive buffers are absorbed.

Hypothetical depth $\log_4 \lambda + O(1)$ gives the least taper $\lambda^{1+\log_4(17/50)+o(1)} \to \infty$. Padding then gives child ratio at most $1/4$ eventually, closing the induction, and sharp ratio $2/9$. The $\mathcal{K}_d$ ratio is now $\gamma = (2/9)\sqrt{50/17}$ and its root coefficient is $\kappa_{\text{low}} = (\pi/\ln 2)\sqrt{100\log_2(162)/33}$. The exact squared inequalities

$$\gamma^2 = \frac{200}{1377} < \left(\frac{31}{81}\right)^2, \qquad \left(\frac{18}{25}\right)^2 < \frac{42}{81}$$

show that $\alpha/(1-\gamma) < 18/25 < \beta$. The same paid recurrence therefore has leading time $\kappa_{\text{low}}\beta$ and space $\kappa_{\text{low}}(2/3)$. All other work stays lower order. □

This is an additional, narrower branch. It must not be advertised as the coefficient for arbitrary $s^{9/10}$-degree inputs. Ordinary bounded-degree SAT encodings satisfy its graph-degree premise for sufficiently large sizes; translating the numerical coefficient to SAT variables still requires the encoding's wire constant.

9 A finite-precision implementation of the known Cauchy basis

The practical implementation targets exactly the feature tables in (14). It supports scalar integer parameters and the quadratic unramified extension

$$R_p = (\mathbb{Z}/2^p\mathbb{Z})[U]/(U^2 + U + 1).$$

An input pair (u, v) represents the exact integer-extension parameter $\zeta = u + vU$. This is a tested concrete specialization, not an implementation of every extension degree required by the full contraction framework.

The recurrence for the products P_b, Q_b is already present in the earlier packet's independent verifier. We do not claim to invent that recurrence. The new contribution is evaluating it safely at finite precision, reusing denominators, and providing a measured implementation with a matched control.

9.1 Why no invalid division of a residue occurs

Each nonzero factor is stored as an exact integer valuation and a unit residue modulo 2^p. A slot difference never has valuation exactly i: it has a lower differing bit, or is divisible by 2^{i+1}. Hence, for nonzero $d = s_j - s_h$,

$$\nu_2(T + d) = \min\{i, \nu_2(d)\} \le i.$$

Split the numerator and denominator before modular reduction. Invert only the unit part of the denominator. Multiplying units uses p-bit residues; normalizing by 2^{E_b} subtracts E_b from the separately tracked exact valuation. The known integrality lemma guarantees a nonnegative remaining valuation. No reduced residue is divided by two, and no discarded low bits are subsequently needed.

For the prefactor $\Gamma(T)/T$, cancel the factor T symbolically first. Split every remaining factor and the scalar $\mathcal{C}'(0)$ denominator into valuations and units. When $\zeta = 0$, the exact zero factor is handled explicitly. The output target/source tables are therefore exactly the prior tables reduced modulo 2^p.

Omit precisely the modes satisfying $w_q \geq p$, using the existing certified weights. Every omitted target is then zero modulo 2^p and every source is integral, so the reconstructed kernel is unchanged. The full-table API can also disable truncation for verification or other callers.

9.2 Reusing inverses and resource cost

Denominators depend only on $d = s_j - s_b$, of which there are $O(L)$ distinct values. Normalize and cache them once. A standard prefix-product batch inverse computes all inverse units with one extension-ring inversion and linear many ring multiplications. The prefactor also uses one separate scalar modular inversion. A separate-inversion strategy is available and is sometimes slightly faster at the small benchmark precisions.

Let B be the number of retained modes, $N = L/2$, and $S_0 = NB - B(B-1)/2$. For nonzero ζ, the instrumented batch variant uses

$$4K + (2L - 1) + 5S_0 - 2B \tag{35}$$

ring multiplications, where K is the number of distinct cached denominators, at most one extension-ring inversion, and one scalar inversion. When $B = 0$ there is no extension-ring inversion. Thus the construction uses $O(NB+L)$ ring operations. Generating/sorting the difference set costs $O(NB+L\log L)$ elementary integer operations. Apart from input normalization and integer bookkeeping, bit work is $O((NB + L)\mathsf{M}(p) + \mathsf{I}(p))$ in this fixed-degree ring.

The table API uses $O(NB + L)$ ring storage, including its output. The streaming API yields one pair of columns at a time using $O(B + L)$ working ring storage, excluding consumer-retained output. Raw input normalization can involve a and the input bit length; only the accumulated unit residues are bounded to p bits. This distinction matters for arbitrary large signed parameters.

9.3 Fair controls

`code/reference_joint_cauchy.py` is the unchanged archived routine. It constructs full exact rational/extension tables with repeated products. The stronger control, `code/reference_recurrence.py`, adapts the quadratic-operation recurrence already in the earlier independent verifier: it supports both arithmetic domains, removes checking from its timed region, and uses exactly the same mode cutoff as the new modular implementation. Both exact controls include conversion to the requested residues. Therefore the main comparison does not force the control to construct discarded modes or execute its verification suite.

10 Measured results and exact verification

The benchmark uses Python 3.12.14 on an AMD EPYC 9V74 host, one warmup and five paired repetitions with alternating method order. All output tables are checked for exact equality. The raw

record contains every wall and process-CPU sample, medians, parameter values, output digests, source hashes, and operation counts. Times are local-kernel observations, not projections from upper-bound constants.

N	p	Domain	B	Exact recurrence (ms)	Modular (ms)	Gain
16	32	Scalar	8	2.427	0.390	6.22×
16	32	Quadratic	8	19.039	0.524	36.35×
32	64	Scalar	16	10.416	1.418	7.35×
32	64	Quadratic	16	76.230	2.271	33.57×
64	64	Scalar	32	43.966	4.541	9.68×
64	64	Quadratic	32	391.070	6.672	58.62×

Table 3: Five-run median wall times for the same finite-precision feature tables and mode cutoff. The modular column uses the batch-inverse option.

For the last quadratic fixture, the archived full exact implementation takes 6.288267 seconds, the stronger recurrence 0.391070 seconds, and the new modular kernel 0.006672 seconds. The approximately 943× ratio against the archived full routine includes recurrence and truncation savings already available in other source files; the 58.62× matched-control comparison is the more informative new result. Across the six fixtures, matched gains range from 6.22× to 58.62×.

These small synthetic cases were selected to exercise the arithmetic domains and growing table sizes. They are not a SAT benchmark distribution. They do not establish a universal speedup over every possible Cauchy implementation. The batch inverse is not consistently faster than separately inverting the same cached units, so the gain is attributed to bounded modular arithmetic and reuse, with batching an optional engineering choice.

10.1 Primary and independent exact checks

The primary verifier passes in ordinary Python and under `python -O`, with identical canonical digests. It covers:

- 1,092 parameter/precision cases and 35,868 independent original-polynomial matrix reconstructions;
- 74,976 target/source comparisons against the archived exact routine and the strengthened recurrence;
- 5,544 streaming-versus-table comparisons, 1,092 batch-versus-separate comparisons, and 600 randomized unit-inversion checks;
- eight invalid-input rejections, signed nodes, zero and nonunit parameters, and both arithmetic domains.

An independent checker evaluates the original cancelled polynomial modulo $2^{p+\nu_2(\mathcal{C}'(0))}$ and only then removes the known scalar denominator valuation. It shares neither the production prefactor nor the feature recurrence. It passes ordinary and optimized Python on 63 further cases, through $N = 128$ and $p = 129$, with 493 reconstructed entries, 493 checks of the public direct-entry API, and 27 full untruncated exact-table comparisons. It includes zero/nonunit parameters and signed 60-bit extension inputs.

The adaptive-aggregation checker separately verifies the partition identity, integer-cost choice, and equality of direct and cached/block contractions on small finite mode graphs, including a ring with zero divisors. The schedule checker verifies exact integer capacity decisions and the coefficient inequalities.

These are implementation and finite-algebra checks; the uniform asymptotic arguments remain the proofs in this report.

The mode-graph suite includes 240 adaptive and 300 streamed comparisons with literal sums, all 280 balanced partitions, 160 exact integer averaging cases, and rational certificates for the safe eventual coefficients 56, 37/35, and 16/15. A separate implementation checks 40 recursive conditioning fixtures over $(\mathbb{Z}/64\mathbb{Z})[V]/(V^2)$, comparing individual leaf-call counts and live assignment contexts as well as exact values. Its results also agree in ordinary and optimized Python.

11 Consequences for SAT and remaining limits

The general and density-sensitive SAT orders already in the repository remain available. Combining the reviewed reductions with the refined engine preserves

$$\mathrm{poly}_k(n,m)\,2^{O_k\left(\frac{n}{\sqrt{\log n}}\min\{\log(2+m/n),\log\log n\}\right)}. \tag{36}$$

As usual, harmless positive shifts may be placed inside the logarithms for bounded n. At $m=O(n)$ this is $2^{O_k(n/\sqrt{\log n})}$; for dense instances it gives (3). Remove unused variables before applying the reductions and multiply the exact count by their power of two. The active variable count is at most $\min\{n, km\}$ for fixed clause width.

The faster cached evaluator changes the time–space coefficient of its normalized contraction engine. It does not prove a denominator $\log n$ in place of $\sqrt{\log n}$. The earlier structured-summation, compact-factorization, and degree-preserving-recursion targets still have missing algorithmic interfaces. Counting a compact index is not the same as cheaply evaluating the associated tensor.

Likewise, a coefficient per wire cannot be inserted directly as a coefficient per SAT variable. The encoding's wire constant, density dependence, reduction branches, original tensor entry costs, and polynomial prefactors all have to be carried through. The alternative manuscript supplies a different big-O theorem with unspecified constants, so no claim of numerical domination over that entire algorithm is justified.

For a resource-aware portfolio, retain the earlier density and lower-degree branches, the polynomial-space algorithm of Section 7, and the new cached branch when memory allows it. Taking the best applicable certificate preserves every reviewed established order and adds the sharper certificates proved here. The practical kernel is usable by either branch.

12 Reproducible deliverable and integration

The archive contains this complete LaTeX source and compiled PDF, standalone standard-library Python code, raw results, independent reviews, a full source inventory, and a build/replay script. The copied reference kernel's original hash and the repository license are included. The project does not require network access to replay the supplied local checks.

The principal integration changes are localized:

1. Retain the existing exact deformation, recovery, and immutable tensor description conventions. Use the sharp precision and full-orientation slope-four basis already present in the reviewed packets.
2. Use asymmetric padding and preserve the separate degree bounds of retained and fresh tensor sides. Implement integer capacity tests and the explicit inward slack, rather than silently using a boundary parameter.

3. For the polynomial-space branch, use balanced binary separator summation and the $r = 16$ schedule in Section 7. For the faster branch, enumerate exact pair-weight arrays, compute their list cardinalities, choose the least crossing product among the 280 triple partitions, build group caches and three block tables, and sum all crossing indices. Pay for each recursive group contraction as in (24).
4. Substitute the modular feature-table implementation where its supported coefficient ring applies. Its exact interface and streaming option are documented in `audit/PRACTICAL_NOTES.md`.

The archive provides a real local arithmetic optimization and a new aggregation prototype. It does not contain a completed end-to-end implementation of the very large upstream SAT framework. The next meaningful engineering milestone is an integrated contraction evaluator with memory-budget selection and creation-precision preservation, followed by paired complete-contraction benchmarks. Another round of abstract parameter retuning alone would not establish a better logarithmic SAT order.

References

[1] OpenAI, *The Exponential Time Hypothesis is False*, October 25, 2026. Pinned source directory: https://github.com/openai/math/tree/7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026. The reviewed files include `build/main_full_version.tex`, `quantitative-contraction.tex`, and `quantitative-reduction.tex`.

[2] Vladimir Reshetnikov and contributors, ProveIt, SAT research snapshot `474ddba696a5d385c091c23fb2995f69edd13e00`: https://github.com/VladimirReshetnikov/ProveIt/tree/474ddba696a5d385c091c23fb2995f69edd13e00/Computability/SubexponentialSAT.

[3] ProveIt, second-round SAT packets, first arrival commit `3e57705cf14ee9ac99c0ac35cfff751dcf70b396`: https://github.com/VladimirReshetnikov/ProveIt/commit/3e57705cf14ee9ac99c0ac35cfff751dcf70b396. Additional arrival `d098ba436b33`; final placement `6d3adc45061535ebe6fa7cd3d9b37dff7dcfc22a`.

[4] *Asymmetric packet balancing for subexponential SAT*, October 26, 2026. Incoming archive `eth_asymmetric_packet_balancing_2026-10-26.zip`; especially `thm:pair-balance`, `cor:padding`, `lem:side-degree`, and its depth-allocation appendix. Immutable archive path and hash are in `provenance/reviewed_sources.json`.

[5] *Full crossing-block Cauchy modes*, October 26, 2026. Incoming archive `eth_joint_cauchy_2026-10-26.zip`; especially `thm:modes`, `sec:compatibility`, `lem:subsetentropy`, and the cached group/block evaluation sections. Exact identities and prototype reference source are credited to this packet.

[6] *Sharper precision for subexponential 3SAT*, `Research/3sat_precision_refinement`; exact taper, precision loss, and full-loss minimax theorem.

[7] *Improving the clause-density dependence of exact k-SAT counting*, `Research/sat_density_improvement`, and *A density-sensitive refinement of exact #k-SAT counting*, `Research/eth_density_refinement`; both in the pinned ProveIt snapshot.

[8] *Subexponential 3SAT asymptotic analysis*, `Research/3sat_asymptotic_analysis`; report and `research_decision/findings.md`, pinned snapshot as above.

[9] *A $2^{o(n)}$ time algorithm for 3-SAT*, credited to Claude (Anthropic), October 26, 2026; filed as `Research/claude-2o-n-3sat/eth.pdf`. Separate manuscript; no supplied matched explicit contraction constant or public formalization build in the reviewed packet.

[10] Adnan Darwiche, "Recursive conditioning," *Artificial Intelligence* 126(1–2), 5–41 (2001). https://doi.org/10.1016/S0004-3702(00)00069-2. The general conditioning technique is prior work; its mode-graph application and explicit certificates are the comparison made here.